

GENERATIVE AI BOOTCAMP

Day 1 • Session 1

Fundamentals of Generative AI

From How LLMs Think to Building Your Own AI Assistants

Instructor: Raghuveer Nishtala

AI Consultant

Comprehensive Session Notes

1. AI Is Not Google: The Mental Model

The session opened with a foundational distinction that reframes how we should think about AI tools: **Google is a search engine, but AI is a context engine.** While both tools can be queried for information, the way they process and return results is fundamentally different.

Google: Keyword Matching

When you type a query into Google (for example, “Best laptops under \$1,000”), Google returns a ranked list of web pages whose content matches those keywords. The user is still responsible for reading through those results, filtering relevant information, and drawing conclusions. Google searches typically work best with short, 4–5 word queries.

AI: Contextual Understanding

When the same query is submitted to an AI tool like ChatGPT, the system draws on its training data to understand the context and intent behind the question. If you specify that you need a laptop for deep learning tasks, or for video editing, or for gaming, the AI can filter its recommendations accordingly, something a traditional keyword-based search cannot do natively.

Key Insight

AI understands human context and natural language. It can interpret your intent, not just your keywords. However, AI may not always be factually correct. It can reference non-authentic sources without the user realizing it. Google is factually neutral (it just ranks pages), while AI may hallucinate.

Knowledge Cutoff and Web Search

Large language models are static models. They are trained once on a massive dataset and retain that snapshot of knowledge. For example, ChatGPT's model at the time of this session had a knowledge cutoff of August 2025. Any question about events beyond that date requires the model to search the internet in real time.

This is why ChatGPT sometimes triggers a web search automatically: it recognizes that the query falls outside the boundary of its training data. However, when it does search, it may pull from sources of varying quality. Raghuvier demonstrated this by searching for Y Combinator startups funded in 2026. The model referenced a website the instructor had never heard of, underscoring the importance of verifying AI-sourced information.

LLM-Based Search Engine Optimization

A new field is emerging around optimizing content to rank in AI-powered search results. ChatGPT is developing capabilities where brands can register their products to appear in AI-generated responses, with a broad rollout expected in 2026.

2. Traditional AI vs. Generative AI

The History of AI

The term “Artificial Intelligence” was coined in 1956 by John McCarthy at the very first AI conference. However, progress stalled for decades due to insufficient computational power, a period known as the AI Winter. The research was there (neural networks, learning algorithms), but the hardware was not.

As computational resources became cheaper and more accessible, AI evolved through waves: machine learning, deep learning, and ultimately generative AI. The pivotal moment came in November 2022, when ChatGPT was released to the public and ignited a global generative AI revolution.

Traditional AI: Behind the Scenes

Traditional AI models were task-specific: one model for spam detection, another for facial recognition, another for object detection. Raghuvver demonstrated this concept using Google's Teachable Machines, a free tool where you can train a binary image classifier (e.g., happy face vs. sad face) without writing any code. He emphasized that these capabilities have existed for years in products we use daily:

- **Gmail spam filters:** classifying emails as inbox-worthy or spam since the mid-2000s
- **Facebook auto-tagging:** recognizing faces in uploaded photos using face classification
- **iPhone photo albums:** grouping images by recognized faces using on-device AI
- **Tesla self-driving:** real-time object detection identifying cars, pedestrians, and obstacles
- **Credit scoring apps:** ML models classifying creditworthiness based on financial data

The key barrier was that building these required technical expertise, namely knowledge of Python, TensorFlow, PyTorch, and deep learning frameworks. Only a handful of trained professionals could build and deploy AI models.

Generative AI: A New Paradigm

Generative AI fundamentally changed the game. Instead of classifying or detecting patterns, generative models create entirely new content (text, images, code, and audio) based on learned patterns. The crucial shift: you no longer need to know programming to build AI-powered applications. You just need to write a prompt.

The Three Pillars of Information

Raghuveer introduced a powerful mental framework: all information on the planet can be conveyed through exactly three types of data, namely text, images (including video, which is simply images at frames per second), and audio. Every AI tool maps to one or more of these pillars:

Text	Images / Video	Audio
ChatGPT, Claude, Gemini, Perplexity	Midjourney, DALL-E, Sora, Veo3	ElevenLabs, Suno.ai

Terminology: LLMs vs. Diffusion Models

Large Language Models (LLMs) handle text-related tasks. The “language” in the name is key: these models specialize in understanding and generating human language. Image and video generation models belong to a different category called diffusion models (e.g., Stable Diffusion, DALL-E, Sora, Veo3).

GPT: What the Acronym Means

GPT stands for Generative Pre-trained Transformer. Generative because it creates new content. Pre-trained because it has been trained on vast amounts of internet data before deployment. Transformer refers to the underlying neural network architecture that makes it all possible.

The AI Startup Boom

Raghuveer noted that 64% of Y Combinator startups in 2025–2026 were AI-focused, a staggering indicator of where investment capital and innovation are flowing. He also noted that approximately 642 text-based LLMs exist as of the session date.

3. How Large Language Models Work

This was one of the most detailed segments of the session. Raghuveer walked the audience through the five-step process that describes how every LLM processes input and generates output, explained at a conceptual level rather than through mathematics.

Step 1: Tokenization

When you type a prompt, the first thing the model does is break your text into tokens. Tokens are numerical representations of words (or sub-words). Raghuveer demonstrated this using OpenAI's Tokenizer tool, showing that the sentence “Explain me about LLM” produces 6 tokens.

Why tokenize? Computers understand numbers, not words. The English language has approximately 400,000–500,000 vocabulary words, which is a finite set. Each word can be assigned a numerical identifier, and the model processes these numbers rather than raw text.

Tokenization and cost: All API pricing in AI is based on tokens. You pay for the number of input tokens and output tokens processed. As a rough guideline, 75 English words equals approximately 100 tokens. Every space, punctuation mark, and special character counts.

Raghuveer showed the OpenAI API pricing page: for GPT-4.1 Mini, the cost is approximately \$0.25 per million input tokens and \$2.00 per million output tokens. He noted that Anthropic's models (Claude) tend to be more expensive, with Claude Opus sitting at approximately \$10 per million tokens.

Step 2: Embeddings

After tokenization, each token is converted into a high-dimensional numerical vector called an embedding. These vectors capture the meaning of words based on how they appear together in the training data.

Raghuveer used an interactive embedding visualization tool to demonstrate: when you type “apple,” the model shows nearby words like “Macintosh,” “Microsoft,” “IBM” (the brand context) as well as “vine” and “fruit” (the food context). The model learns these associations from the billions of sentences it has processed during training.

He showed that ChatGPT's embeddings operate in 1,536 dimensions, far beyond the 3D space humans can visualize. When asked to output the embedding for “hello world,” ChatGPT returned a massive string of numbers representing that phrase's position in its semantic space.

The “Apple” Test

If you type “market cap of Apple,” the model knows you mean the company. If you type “how many calories in apple,” it knows you mean the fruit. But if you type just “I like apple,” the model gets confused and may ask for clarification. This demonstrates how context determines meaning in embeddings.

Step 3: Self-Attention

The transformer mechanism at the heart of every LLM uses a process called self-attention to determine which parts of your input are most important for generating the response. This is the reason prompting matters so much.

Raghuveer demonstrated this dramatically: he asked ChatGPT “Explain me about LLM” and received a highly technical response filled with jargon. Then he asked the exact same question but added “as if I am 5 years old,” and the response transformed into a playful analogy about a smart parrot who has read millions of books. The self-attention mechanism

focused on that additional context and completely reshaped the output.

Step 4: Prediction

LLMs are fundamentally prediction machines. They predict the next token, one at a time, based on all the preceding tokens and the attention weights computed in the previous step.

Raghuveer used the classic example: “The cat sat on the ____.” Out of 500,000 vocabulary words, maybe 20–50 could fit. The model assigns probabilities: “mat” might get 0.6, “floor” might get 0.4, and “democratic” might get 0.000001. It selects the highest-probability token and then predicts the next one, and so on.

Step 5: Response Generation

The model continues predicting one token at a time until the response is complete. This is why speed matters: the metric “tokens per second” determines how fast a model responds. Raghuveer showed the ArtificialAnalysis.ai leaderboard, where models are benchmarked on intelligence, speed (output tokens per second), and pricing per million tokens.

Why the Same Question Gets Slightly Different Answers

Because LLMs are probabilistic, asking the same question twice will yield similar but not identical responses. The core ideas align, but the specific word choices and examples may vary, because the model samples from its probability distribution each time.

4. The Art and Science of Prompting

With the foundation of how LLMs work established, Raghuveer transitioned to the practical skill of prompting: the discipline of writing structured instructions that guide AI to produce better, more personalized output.

4.1 The Magic Prompt Formula (6-Point Checklist)

The first framework covered was the six-element prompt checklist. Raghuveer stressed that three of these six are mandatory (marked with a star), while the other three are good to have:

Element	Required?	Description
Persona / Role	■ Mandatory	Give the AI a specific role (e.g., “You are a fitness and nutrition coach expert”). This narrows the knowledge base the model draws from, like walking into the correct section of a library.
Task	■ Mandatory	State your high-level objective in 1–2 lines (e.g., “Create a 7-day diet and exercise plan”).

Context	■ Mandatory	Provide all relevant background information. Without context, the AI will assume details about you, and assumptions lead to generic, unhelpful output.
Examples	Optional	Show the AI what good output looks like. Provide sample formats, reference styles, or previous work to guide the response pattern.
Format	Optional	Specify how you want the output delivered: tables, bullet points, paragraphs, Word document, PDF, etc.
Tone	Optional	Define the communication style: conversational, professional, inspiring, friendly, academic, etc.

Live Demo: The Fitness Planner

To illustrate the power of this framework, Raghuveer built a live “Ria” fitness chatbot prompt. In a bad prompt scenario, typing “I want a diet plan and exercise plan, I am male, 30 years old, vegetarian” produced a generic response that assumed the user wanted gym workouts, five daily meals, Greek yogurt (without checking for lactose intolerance), and a standard exercise split, none of it personalized.

Using the six-point formula, Raghuveer crafted a prompt that assigned a persona (fitness and nutrition coach), defined the task, and critically, instructed the AI to **ask contextual questions** before generating the plan. The instruction “ask me one question at a time” transformed the experience into a conversational interaction. The AI asked about weight, height, profession, exercise preferences, allergies, supplements, medical conditions, goals, and meal preferences before generating a fully personalized 7-day plan in table format.

The Coffee Shop Analogy

Raghuveer played a humorous video of a man ordering coffee who gets annoyed by the barista's detailed questions (size? milk type? ice level? drizzle? lid type?). The punchline: that level of detail is annoying for humans but is exactly what AI needs to produce precise, personalized output.

Prompting Rules

Rule 1: Set clear expectations. Use precise keywords like “research thoroughly,” “think step by step,” or “give me a brief answer quickly.” These phrases are backed by research and materially change how the model generates its response.

Rule 2: Avoid mixed messages. Do not start a prompt asking for “creative ideas” and end it saying “stick to proven methods.” Contradictory instructions confuse the model. Similarly, do not request both a “detailed explanation” and a “quick response” in the same prompt.

4.2 Chain of Thought Prompting

Raghuveer introduced Chain of Thought (CoT) prompting, a technique published by the Google Research Brain Team. The core discovery: simply adding the phrase “Let's think step by step” to a prompt can dramatically improve accuracy on reasoning tasks.

He demonstrated with a classic example from the original research paper: a word problem about buying and giving away apples, where the model gave the wrong answer without CoT but produced the correct answer (10) when prompted with “let's think step by step.” This technique is now the foundational backbone of all “thinking” models. Every reasoning model (ChatGPT's thinking mode, Claude's extended thinking) uses chain of thought by default.

Raghuveer recommended promptingguide.ai as the comprehensive Bible for all prompting techniques, but emphasized that for 90% of tasks, you only need two or three techniques: the six-point formula, CoStar, and chain of thought.

4.3 The CoStar Framework

The second formal prompting framework covered was CoStar, an acronym that builds on the magic prompt formula by adding structured steps and a self-reflection mechanism:

- **C (Context):** Background information about the task and the user
- **O (Objective):** The specific goal or task to accomplish
- **S (Steps):** A step-by-step breakdown of how to accomplish the task (chain of thought)
- **T (Tools / Data Formats):** Specify any tools, formats, or data the AI should use
- **A (Audience):** Who the output is for (adjusts tone and complexity)
- **R (Reflection):** Instruct the AI to self-check before finalizing output

The reflection step is what sets CoStar apart. It instructs the AI to verify that all required information has been collected, ensure recommendations align with the user's stated goals, and ask follow-up questions if critical information is missing, before producing the final output.

Live Demo: Finance Planner with CoStar

Raghuveer demonstrated the CoStar framework by building a finance planner prompt. He typed a description of what he wanted into ChatGPT and asked it to generate a prompt using the CoStar framework. The resulting prompt included structured steps (financial discovery, risk assessment, goal identification) and a reflection section that instructed the AI to verify all user information before finalizing its asset allocation recommendation.

When this prompt was used in practice, the AI systematically asked about income, expenses, active loans, dependents, marital status, home ownership, insurance, risk appetite, investment horizon, and financial goals before producing a personalized allocation strategy.

4.4 Visual Prompting

The final prompting technique covered was visual prompting: the ability to feed images into an AI model and prompt it for analysis. Raghuveer demonstrated this by uploading a complex financial dashboard screenshot to ChatGPT and prompting it to extract key insights, identify risks, and flag areas requiring caution.

The AI analyzed the dashboard's KPIs, identified trends in receivables and payables, calculated ratios (DSO, DSI, DPO), assessed working capital trends, and provided a structured risk analysis, all from a single image. Raghuveer emphasized that visual prompting works across all major AI platforms (ChatGPT, Claude, Gemini, Perplexity) and that prompting techniques are platform-independent.

5. System Prompts vs. User Prompts

One of the most practically important concepts of the session was the distinction between system prompts and user prompts: the foundation for building reusable AI assistants.

The Iron Man Analogy

Raghuveer used a memorable movie analogy: think of Tony Stark's character in Iron Man. The character description (a self-absorbed billionaire genius who masks insecurities with humor, charming but arrogant, brilliant but reckless) remains consistent throughout the entire film. That character definition is the system prompt. The scenes change (weapons demo, cave capture, press conference), and Tony's dialogue changes with each scene, but his core personality never does. The scenes represent user prompts.

How It Works Technically

System Prompt: A fixed set of instructions that defines the AI's personality, role, behavior rules, knowledge boundaries, and output format. It is set once and remains constant across all interactions. Example: "You are a helpful financial advisor who explains complex topics in simple terms. Never give risky advice, and always remind people to consult professionals."

User Prompt: The variable input that changes with each interaction. Different users can ask different questions ("What is a SIP?" "Should I invest in crypto?" "Should I buy gold?"), and the AI responds according to its system prompt's personality and constraints every time.

If the system prompt defines a financial advisor and a user asks for a diet plan, the AI will politely decline: "I am a financial advisor, I can't help with diet plans." This boundary enforcement comes from the system prompt.

6. Memory Instructions on Claude & ChatGPT

If a system prompt is how you shape the AI's behavior, **memory instructions are how you shape what the AI knows about you.** Raghuveer reframed the most common frustration

with AI tools: people re-type the same background (“I’m a 30-year-old marketing manager in Bangalore, I prefer concise answers...”) at the start of every single chat. The fix is to tell the AI once, save it, and have every future conversation start already personalized.

Both ChatGPT and Claude let you store a persistent profile that loads automatically before every message. The principle ties directly back to the session's running theme, **context is where you shine**: instead of supplying context per prompt, you supply it once at the account level.

6.1 ChatGPT: Custom Instructions & Memory

In ChatGPT, open **Settings → Personalization → Custom Instructions**. There are two boxes. The first, “What would you like ChatGPT to know about you?”, is for your background. The second, “How would you like ChatGPT to respond?”, is for your communication preferences. Anything you write here is applied to all new conversations automatically and is available even on the free tier.

ChatGPT additionally has a separate **Memory** feature that automatically remembers useful facts it picks up during chats (for example, that you are preparing for a marathon, or that you manage a 5-person team). You can review, edit, or delete individual memories, or turn the feature off entirely, from the same Personalization settings.

6.2 Claude: Profile Instructions

In Claude, click your initials in the lower-left corner, then go to **Settings → Profile** and fill in the personal preferences field (sometimes labelled “custom instructions”). Whatever you put here becomes account-wide context that Claude reads at the start of every conversation. Styles, a separate control, then adjust only *how* the response is formatted (concise, explanatory, formal). The three layers stack: profile instructions load first, project instructions add context, and styles adjust delivery.

Your Personal Context Template

Whether you are filling ChatGPT's box or Claude's profile, cover these points so the AI never has to guess: **(1) Who you are** – name, role, company/industry, city. **(2) Expertise level** – beginner vs. expert in your domain, so explanations land at the right depth. **(3) What you do repeatedly** – the tasks you use AI for most. **(4) Goals** – what you are currently working toward. **(5) Communication preferences** – length, tone, formatting, language. **(6) Constraints & do-nots** – tools you use, things to always or never do (e.g., “always give sources,” “never flatter, challenge my assumptions”). Keep it focused, roughly under 500 words, since this text is consumed on every single message.

6.3 Going Deeper: Claude Projects

Profile instructions are global, they apply to *everything*. But much of your context is situational: the brand guidelines for one client, the financial profile for your personal planning, the codebase conventions for one app. This is exactly what **Claude Projects** are

built for, and Raghuveer recommended them as the single most powerful personalization feature for anyone who uses Claude regularly for the same kind of work.

A Project is a persistent workspace with three parts working together. Think of it as the no-code equivalent of deploying Claude with your own custom system prompt (the concept from Section 5), now combined with a private knowledge base:

- **Project instructions** – a custom system prompt that Claude reads at the start of every conversation inside that project, separate from your global profile.
- **Project knowledge** – documents you upload once (your resume, a brand guide, a financial profile, a glossary, examples of your best work). Claude automatically pulls the relevant excerpt into any chat in the project, so you stop re-pasting context.
- **Scoped conversation history** – chats inside the project can see each other, so context compounds over time instead of resetting every session.

To create one: go to claude.ai, click **Projects** in the sidebar, choose **Create project**, give it a name and short description, paste a focused set of project instructions (a 200–500 word block covering profile, voice, rules, and formatting), then upload two or three reference documents as project knowledge. Projects are available to all users, with free accounts able to create up to five.

What goes in knowledge vs. chat: good knowledge files are stable reference material, your style guide, reusable templates, a personal background document, domain glossaries. Keep one-off, confidential, or constantly-changing files in the individual chat instead, not in the long-lived workspace.

Homework Assignment from the Session

Raghuveer challenged participants to stop re-explaining themselves to AI. First, write your personal context once into ChatGPT's Custom Instructions and Claude's Profile. Then build one Claude Project for a domain you work in repeatedly (personal finance, marketing, a specific client, your codebase): give it sharp instructions, upload your key reference documents, and run all related conversations inside it. Write the context once, reuse it forever.

7. Canvas & Artifacts: Building Apps with AI

The session's penultimate segment covered the collaborative coding and writing features now available in ChatGPT (Canvas) and Claude (Artifacts).

ChatGPT Canvas

Canvas is a collaborative editor that opens a side panel where you and the AI co-create content in real time. Unlike the standard chat interface where you prompt and receive, Canvas lets you directly edit the AI's output: selecting specific sections to modify, requesting

targeted changes, and adding your own content alongside AI-generated text.

Raghuveer demonstrated by creating a blog post about “The Future of the Workforce with AI Agents.” Canvas generated the full article with headers and sections. He then selected a specific section and instructed the AI to add more examples of how AI agents increase productivity. Only that section changed, while the rest of the article remained untouched. He also showed how to add conclusions, adjust reading level, add emojis, and download the final output as a Word document.

Claude Artifacts

Artifacts in Anthropic's Claude go a step further: they can render live, interactive HTML applications directly in the interface. Raghuveer demonstrated this by building a full Finance Planner web application from a single prompt. Claude's artifact system asked clarifying questions about the type of app, visual style, and functionality, then generated a complete multi-step form with fields for personal profile, financial profile, liabilities, life goals, and risk assessment.

The resulting app was fully interactive. You could fill in all the fields and click “Generate My Plan” to receive an AI-powered asset allocation recommendation. Raghuveer noted that Claude Artifacts can be published with a public link, making them shareable. He also mentioned that Google offers similar functionality through Google AI Studio (not Gemini directly).

8. AI Agentic Browsers: Text In, Actions Out

The final demonstration of the session showcased the cutting edge of AI capability: **agentic browsing** that doesn't just generate text but takes real-world actions on your behalf, right inside the browser you already use.

Raghuveer demonstrated the **Claude for Chrome** extension (by Anthropic), which lets Claude see and act on web pages in your browser: navigating sites, clicking buttons, filling forms, and extracting information across multiple tabs. He instructed it: “Go to LinkedIn, search for Applied Scientist jobs matching 7–8 years of experience, and prepare a Google Sheet with company names and links.” The extension autonomously navigated LinkedIn, applied filters, scanned job listings, and compiled the results into a Google Sheet, with a glowing border around the screen indicating Claude was at work.

He showed a previous task where he had asked it to compile a list of Y Combinator 2026-funded startups into a spreadsheet, complete with company names, platform descriptions, industries, locations, batch information, and website links, all gathered automatically. Because the extension runs in your own Chrome, it works on any site you are already signed into and has built-in knowledge of common platforms like Gmail, Google Calendar, Google Docs, Slack, and GitHub.

Installing the Claude for Chrome Extension

Claude for Chrome is a beta browser extension available on all paid Claude plans. It only works in Google Chrome on desktop, not other Chromium browsers or mobile. Set it up as follows:

1. Make sure you are signed in to a paid Claude plan. On Pro the extension runs on the Haiku model; Max, Team, and Enterprise plans can choose more powerful models.
2. Open Google Chrome and go to the Chrome Web Store. Search for “**Claude**” and select the extension published by **Anthropic** (avoid third-party look-alikes).
3. Click **Add to Chrome**, then confirm with **Add extension** in the pop-up.
4. Pin it for easy access: click the puzzle-piece icon in the Chrome toolbar, then the thumbtack next to Claude.
5. Click the Claude icon to open the side panel, sign in with your Claude account, and grant the requested permissions.
6. Describe a task in plain English, or record a workflow once to teach Claude how to repeat it.

Raghuveer recommended starting in “Ask before acting” mode, where Claude proposes a plan and waits for your approval before executing. Even with broader permissions, it still pauses before irreversible or high-risk actions such as making a purchase. These browser tasks, such as trip planning, LinkedIn engagement, job applications, and competitive research, are the same kinds of multi-step work demonstrated throughout the session, now executed end-to-end.

Use It Safely

Browser AI can encounter prompt injection, hidden instructions on a web page that try to hijack Claude's actions. Anthropic has built defenses, but Raghuveer stressed sensible habits: start by granting access only to trusted, familiar sites; always review before Claude handles anything financial, personal, or work-critical; and report unexpected behavior through the feedback options.

Key Takeaways

1. AI is a context engine, not a search engine.

It understands intent, not just keywords. But always verify AI-sourced information, as hallucination remains a risk.

2. LLMs work in 5 steps.

Tokenization → Embeddings → Self-Attention → Prediction → Response Generation.
Understanding this helps you write better prompts.

3. Prompting is the new coding.

The six-point formula (Persona, Task, Context, Examples, Format, Tone) and the CoStar framework are all you need for 90% of tasks.

4. Context is where you shine.

Without your context, AI will assume, and assumptions produce generic output. The more relevant detail you provide, the more personalized the result.

5. Context can be permanent.

Save your background once in Custom Instructions / Profile, and a Claude Project for focused work. Every conversation then starts personalized, no more re-explaining yourself.

6. Build, don't just chat.

Canvas and Artifacts let you co-create documents and build fully interactive web applications directly from prompts. Claude Artifacts can even be published and shared.

7. AI agents take actions.

The Claude for Chrome extension can navigate websites, fill forms, compile spreadsheets, and complete multi-step tasks autonomously, right inside your browser.